Neural SC Decoder for Gaussian MAC Polar Codes: Problem Statement and Request for Ideas

Project Documentation for External Collaboration

March 2026

1 Background

1.1 Two-User MAC Polar Codes

Polar codes, introduced by Arıkan [1], achieve channel capacity through a phenomenon called *polarization*: as block length $N = 2^n$ grows, each synthetic sub-channel becomes either perfectly reliable or perfectly noisy. For a two-user multiple-access channel (MAC), Önay [2] showed that polar codes can simultaneously achieve the capacity region by having each user independently encode with a polar code, while a joint successive cancellation (SC) decoder processes the superimposed codeword.

1.2 Channel Model: Gaussian MAC

We consider a two-user Gaussian MAC where users X and Y transmit BPSK-modulated polar codewords. The received signal is:

$$Z = (1 - 2X) + (1 - 2Y) + W, \qquad W \sim \mathcal{N}(0, \sigma^2), \tag{1}$$

where $X, Y \in \{0, 1\}$ are coded bits and the per-user SNR is $\mathrm{SNR} = 1/\sigma^2$. The observation $Z \in \mathbb{R}$ is continuous, unlike the binary-erasure MAC (BEMAC) where $Z = X + Y \in \{0, 1, 2\}$.

At each leaf position t of the decoding tree, the decoder must compute the posterior over the four joint hypotheses $(u_t, v_t) \in \{00, 01, 10, 11\}$, where u_t and v_t are the t-th information bits of users X and Y respectively.

1.3 Decoding Paths and Code Classes

The joint SC decoder processes $2N$ bits total (N per user) in a specific interleaving order called a *path*. Let π_i index the path, with $0 \le \pi_i \le N$. Three classes arise:

- **Class A** ($\pi_i = 0$): Decode all of Y first, then all of X. An *extreme* path.
- **Class C** ($\pi_i = N$): Decode all of X first, then all of Y. Also extreme.
- **Class B** ($\pi_i = N/2$): Interleave X and Y decisions. A *monotone chain* path.

Classes A and C are symmetric and use only `CalcLeft` and `CalcRight` (downward tree operations). Class B additionally requires `CalcParent`—an *upward* operation that propagates decided information back up the tree before descending into the next user's sub-tree. This is the key algorithmic difference.

1.4 Complexity and Motivation

The analytical SC decoder for the two-user MAC has complexity:

$$\mathcal{O}(m^2 \cdot N \log N), \tag{2}$$

where $m = 4$ is the joint alphabet size $|\{00, 01, 10, 11\}|$. Each tree operation manipulates probability tensors of size m^ℓ at depth ℓ, but the $\mathcal{O}(N \log N)$ computational graph structure keeps the total cost manageable.

For channels *with memory* (e.g., inter-symbol interference, fading), each tree operation must track a state space of size S, giving complexity:

$$\mathcal{O}(S^3 \cdot N \log N). \tag{3}$$

Even moderate state spaces ($S = 16$) make this prohibitive. Our goal is to replace the $\mathcal{O}(S^3)$ tensor operations with fixed-size neural networks of complexity $\mathcal{O}(md)$, where d is the embedding dimension:

$$\mathcal{O}(S^3 \cdot N \log N) \;\longrightarrow\; \mathcal{O}(md \cdot N \log N). \tag{4}$$

This would yield a **channel-independent decoder**: the tree operations are learned once and reused across any channel, with only the front-end embedding layer changing per channel.

2 Architecture: Neural Computational Graph (NCG) Decoder

2.1 Overview

The NCG decoder replaces each analytical tree operation with a small MLP, while preserving the exact SC decoding schedule (the order in which tree nodes are visited). Every edge in the binary tree stores a d-dimensional embedding vector instead of a probability tensor. All MLPs share weights across tree levels, making the decoder **independent of block length** N.

2.2 Tree Structure

The binary tree has $N-1$ internal vertices and N leaves. Edges are indexed $1, \ldots, 2N-1$:

- Edge 1 = root (channel observation embedding).
- Edges $N, \ldots, 2N-1$ = leaves (one per bit position).
- Vertex β has parent edge β, left child edge 2β, right child edge $2\beta+1$.

```
            [Edge 1: root]          <-- z_encoder output (d=16)
           /              \
     [Edge 2]            [Edge 3]    <-- CalcLeft / CalcRight (3d -> d)
     /      \            /      \        CalcParent goes UP (2d -> d)
  [Edge 4] [Edge 5] [Edge 6] [Edge 7]
     |        |        |        |
   Leaf 0   Leaf 1   Leaf 2   Leaf 3   <-- Emb2Logits (d -> 4 classes)
```

Figure 1: NCG tree for $N = 4$. Each edge stores a d-dimensional embedding.

2.3 Neural Operations

1. **CalcLeft** (top-down, $\mathbb{R}^{3d} \to \mathbb{R}^d$): Given parent embedding $\mathbf{e}_\beta$ and child embeddings, compute left child's top-down message. Input is concatenation $[\mathbf{e}_\beta \| \mathbf{e}_{2\beta} \| \mathbf{e}_{2\beta+1}]$. Two-layer MLP with ReLU.
2. **CalcRight** (top-down, $\mathbb{R}^{3d} \to \mathbb{R}^d$): Same structure as CalcLeft but separate weights. Computes right child's top-down message.
3. **CalcParent** (bottom-up, $\mathbb{R}^{2d} \to \mathbb{R}^d$): Propagates a decision back *up* the tree. Takes child embedding (after decision) and sibling embedding, produces updated parent embedding. Uses a **gated residual MLP**: $\mathbf{e}_{\text{out}} = \alpha \cdot \text{MLP}([\mathbf{e}_{\text{child}} \| \mathbf{e}_{\text{sibling}}]) + (1-\alpha) \cdot \mathbf{e}_{\text{parent}}$, where $\alpha = \sigma(\text{gate}(\cdot))$.
4. **Emb2Logits** ($\mathbb{R}^d \to \mathbb{R}^4$): At each leaf, converts embedding to 4-class log-probabilities over $(u, v) \in \{00, 01, 10, 11\}$.
5. **Logits2Emb** ($\mathbb{R}^4 \to \mathbb{R}^d$): After a leaf decision, converts the decided log-probabilities back to an embedding for tree propagation.

2.4 Channel Front-Ends

The front-end converts channel observations into root embeddings:

- **BEMAC**: `nn.Embedding(3, d=16)` — discrete lookup for $Z \in \{0, 1, 2\}$.
- **GMAC**: `MLP`$(1 \to 32 \to 16)$ — continuous mapping for $Z \in \mathbb{R}$.

The GMAC front-end must learn to map a continuous Gaussian observation to the same embedding space that the tree operations expect. This is a key difficulty.

2.5 Training

- **Teacher forcing**: At training time, leaf decisions use ground-truth (u_t, v_t) to avoid error propagation. Loss is cross-entropy on the 4-class output at each leaf.
- **Curriculum learning**: Essential for convergence. Train at $N = 8$ from scratch, then fine-tune at $N = 16 \to 32 \to 64 \to \cdots \to 1024$.
- **Total parameters**: ~39K (all shared across tree levels).
- **Training cost**: ~1 second/iteration at $N = 128$ on CPU; ~4 seconds at $N = 1024$.

2.6 Soft-Bit Bridge (for CalcParent)

For the BEMAC channel, we found that a pure neural CalcParent fails. Instead, we use an analytical *bridge*: convert embedding to logits (Emb2Logits), apply analytical circular convolution in probability space (differentiable), then convert back (Logits2Emb). This preserves exact information flow while keeping end-to-end differentiability.

3 Results: What Works and What Doesn't

3.1 BEMAC Results (Discrete Channel, $Z = X + Y$)

The NCG decoder with Soft-Bit Bridge for CalcParent achieves remarkable results on the discrete BEMAC:

Table 1: BEMAC Class B: NCG vs. Analytical SC (SNR matched, proper frozen sets)

N	NN BLER	SC BLER	Ratio
16	0.011	0.011	1.0× (matches)
32	0.009	0.008	1.1×
64	0.003	0.006	0.5× **(NN wins)**
128	0.001	0.002	0.6× **(NN wins)**
512	0.0004	0.0004	1.0× (matches)
1024	0.0003	0.0005	0.6× **(NN wins)**

The NN **matches or beats SC at every tested block length**. The architecture and training methodology are sound. The Soft-Bit Bridge for CalcParent works perfectly for discrete channels.

3.2 Neural SCL: List Decoding on Top of Neural Tree (NEW)

A major breakthrough: we implemented **Neural SC-List (SCL) decoding** on top of the neural tree operations. Instead of greedy argmax at each leaf, the decoder maintains L candidate paths through the tree, forking at each non-frozen leaf into up to $2L$ candidates (one per bit hypothesis, marginalising over the other user) and keeping the best L by cumulative log-probability.

Table 2: BEMAC Class B: NN-SCL ($L = 4$) vs. SC and SCL-32

N	SC	SCL-32	NN-SC	NN-SCL (L=4)	Ratio to SC
32	0.0080	0.0082	0.0088	0.0073	0.92×
64	0.0056	0.0010	0.0030	0.00067	**0.12× (8.4× better)**
128	0.0020	0.0006	0.0012	0.00067	**0.33× (3× better)**

Key findings:

- At $N = 64$, NN-SCL ($L = 4$) achieves BLER $= 6.7 \times 10^{-4}$, which is **8.4× better than SC** and **1.5× better than SCL-32**.
- At $N = 128$, NN-SCL matches SCL-32 performance (both $\sim 6 \times 10^{-4}$), and is **3× better than SC**.
- The neural tree operations learn a richer representation than the analytical probability tensors, which the list decoder can exploit more effectively.
- This is the **first demonstration of neural list decoding for MAC polar codes** outperforming both greedy neural and classical analytical decoders.

3.3 GMAC Results (Continuous Channel, SNR = 6 dB)

The same architecture with a continuous front-end fails dramatically at large N:

Table 3: GMAC Class B: NCG vs. Analytical SC (SNR = 6 dB)

N	NN BLER	SC BLER	Ratio
32	0.040	0.046	0.87× **(NN wins)**
64	0.031	0.025	1.25×
128	0.032	0.015	2.1×
256	0.025	0.005	5.0×
512	0.045	0.001	45×
1024	0.069	0.0003	230×

The NN BLER stays flat at ∼0.03–0.07 regardless of N, while SC BLER drops exponentially (the hallmark of polarization). The gap grows exponentially: 230× at $N = 1024$.

3.4 Interpretation

The contrast is stark. On BEMAC (3-valued discrete input), the NN captures polarization perfectly. On GMAC (continuous input), it does not. The only architectural difference is the front-end: `nn.Embedding(3, 16)` vs. `MLP(1, 32, 16)`. The tree operations are identical.

This suggests the continuous front-end fails to produce embeddings with sufficient *precision* for the tree operations to exploit polarization at large N. Polarization requires that information accumulates multiplicatively through $\log N$ tree levels. Any imprecision in the root embedding is amplified exponentially.

4 What We've Tried (and Why It Failed)

4.1 More Training Time

Diminishing returns. At $N = 1024$, each iteration takes $\sim$4 seconds (2048 sequential MLP calls through the tree). We managed $\sim$4K iterations. At $N = 32$, we ran 30K iterations. The loss converges but BLER does not improve beyond the plateau.

4.2 Knowledge Distillation from Analytical SC

Three variants attempted:

1. **Phase-based** (freeze all except CalcParent, train CalcParent from teacher): The z_encoder never trains because it is frozen. Wasted iterations.
2. **MSE on all intermediate embeddings**: Teacher and student learn different embedding spaces. MSE loss exploded to 10^{18}.
3. **Clamped MSE**: Stable training but no improvement over baseline (BLER 0.072 vs. 0.070).

Why it fails: The analytical decoder operates on probability tensors in $\mathbb{R}^{4^\ell}$ at depth ℓ. The neural decoder uses $\mathbb{R}^d$ embeddings. There is no natural correspondence between these representation spaces.

4.3 BEMAC→GMAC Transfer Learning

Freeze the pre-trained BEMAC tree weights (which work perfectly); train only the GMAC front-end.

- $N = 32$: Works (BLER = 0.095 in 8K iters).
- $N = 64$: Fails (loss stalls at 0.81, near random).

Why it fails: The tree operations are not truly channel-independent. At larger depths, the embeddings carry channel-specific structure that the GMAC front-end cannot replicate.

4.4 Fast Cross-Entropy (Auxiliary Loss at Internal Edges)

Following the NPD literature [4], we added auxiliary cross-entropy losses at every internal tree edge (not just leaves). This provides denser gradient signal.

Result: Helps at $N = 32$, hurts at $N \geq 64$. Internal edges have ambiguous targets when the tree walk is path-dependent.

4.5 Architecture Variations

- Embedding dimension d: tested 8, 16, 32, 48. No significant difference.
- Front-end hidden size: tested 16 to 128. No difference.
- Per-level weights (separate MLPs per tree depth): Needs 10× more training data, untested at scale.
- Temperature scaling ($\tau = 1.0, 1.5, 2.0$): $\tau = 1.0$ is optimal; higher τ hurts.

4.6 Training Tricks

- **Label smoothing**: Helps from-scratch training but not with curriculum.
- **Scheduled sampling**: Good training loss but BLER = 1.0 at test time. The model becomes dependent on noisy inputs during training and cannot recover.
- **Multi-SNR training**: Not yet tested. Could help generalization.

4.7 Pure Neural CalcParent (No Soft-Bit Bridge)

Attempted a GRU-gated MLP for CalcParent with knowledge distillation from the analytical bridge:

- Phase A (teacher available): BLER = 0.012. Works.
- Phase B/C (teacher removed): Collapses to BLER = 1.0.

The neural CalcParent cannot maintain information fidelity without the analytical bridge.

5 Analysis and Open Questions

5.1 The Core Problem: Failure to Learn Polarization

The defining feature of polar codes is *polarization*: as $N \to \infty$, the fraction of "good" sub-channels approaches the channel capacity, and each good sub-channel becomes arbitrarily reliable. The analytical SC decoder exploits this: its BLER drops as $\mathcal{O}(2^{-N^{0.5}})$.

Our neural decoder's BLER stays flat. It does not learn to exploit the exponentially increasing reliability of polarized sub-channels. This is not an architecture problem per se—the BEMAC results prove the architecture works—but a *representation problem* specific to continuous channels.

5.2 Gradient Analysis

At $N \geq 64$, we observe:

- **z_encoder (front-end)**: Receives near-zero gradients. The 10+ tree levels between the root and the leaves create a severe **vanishing gradient** problem.
- **CalcParent**: Near-zero gradients (same reason).
- **Emb2Logits (leaf head)**: Receives most of the gradient. But it can only learn the final mapping, not fix upstream representation issues.

For BEMAC, this is not an issue because the discrete embedding lookup (`nn.Embedding`) needs minimal gradient—it only has 3 entries. For GMAC, the continuous MLP front-end needs substantial gradient to learn the mapping from $\mathbb{R}$ to $\mathbb{R}^d$, and it never receives it.

5.3 Key Discovery: CalcParent Call Counts

We initially hypothesized that Class B fails because it requires more CalcParent calls (bottom-up operations). Careful counting revealed:

- $N = 1024$, Class C: 2035 CalcParent calls.
- $N = 1024$, Class B: 2033 CalcParent calls.

The counts are nearly identical. **CalcParent frequency is not the bottleneck.** The problem lies elsewhere—likely in the continuous front-end's inability to produce sufficiently precise embeddings.

5.4 Specific Questions

1. **Injecting polarization structure**: Is there a way to encode the polarization hierarchy into the neural architecture? For example, can we use different precision levels at different tree depths, or use a loss function that explicitly rewards polarization-like behavior?
2. **Loss function redesign**: Cross-entropy treats all bit errors equally. But in SC decoding, an error at position t causes cascade errors at positions $t+1, \ldots, 2N$. Can we design a loss that weights early positions more, or that captures error propagation?
3. **Alternative architectures**: Should we abandon the tree-walk structure for continuous channels? A transformer that processes the full observation sequence $Z_1, \ldots, Z_N$ autoregressively might handle continuous inputs better. We tried this for BEMAC Class B at $N = 8$ (matched SC) but it failed at $N = 16$ due to exposure bias.
4. **Single-user neural decoders**: The NPD literature (CRISP [4], DeepPolar [5]) achieves near-ML decoding for single-user polar codes. Can their techniques (e.g., multi-round belief propagation, learned code construction) be extended to the two-user MAC?
5. **SCL on top of NN** (**RESOLVED**): We implemented Neural SCL ($L = 4$) on top of the neural tree and it works *spectacularly* for BEMAC: 8.4× better than SC at $N = 64$. However, for GMAC the neural tree embeddings are still imprecise at large N, so the SCL gain is limited. The question shifts to: how can we make the continuous front-end produce sufficiently precise embeddings for the neural SCL to exploit?
6. **Is shared-weight the bottleneck?**: All CalcLeft/CalcRight/CalcParent MLPs share weights across tree levels. Does the continuous channel require level-specific parameters? If so, can we achieve this without $\mathcal{O}(n)$ parameter growth (e.g., via hypernetworks or FiLM conditioning on depth)?

5.5 Available Resources

- Mac with 8 GB RAM (CPU training, ∼1 second/iteration at $N = 128$).
- Google Colab (T4 GPU, 12-hour free sessions).
- Complete analytical SC decoder for GMAC (can generate unlimited training data).
- Pre-trained BEMAC NCG decoder that matches/beats SC up to $N = 1024$.
- Pre-trained GMAC NCG decoder (works at $N = 32$, fails at $N \geq 64$).
- Approximately one week of compute budget.

5.6 Summary

We have a neural SC decoder architecture that **provably works** for discrete MAC channels (BEMAC) at all block lengths up to $N = 1024$. Furthermore, **Neural SCL** ($L = 4$) achieves dramatic gains over both SC and SCL-32 for BEMAC: 8.4× better than SC at $N = 64$, marking the first successful neural list decoder for MAC polar codes. The same architecture fails for continuous channels (GMAC) at $N \geq 64$. The failure mode is a flat BLER curve that does not benefit from polarization. Gradient analysis points to vanishing gradients through the tree as the proximate cause, and the continuous front-end's inability to produce high-precision embeddings as the root cause. We seek architectural or training innovations that can bridge this gap—particularly for the continuous front-end, so that Neural SCL can deliver similar gains on GMAC as it does on BEMAC.

References

[1] E. Arıkan, "Channel polarization: A method for constructing capacity-achieving codes for symmetric binary-input memoryless channels," *IEEE Trans. Inf. Theory*, vol. 55, no. 7, pp. 3051–3073, Jul. 2009.

[2] S. Önay, "Successive cancellation decoding of polar codes for the two-user binary-input MAC," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Jul. 2013, pp. 1532–1536.

[3] Y. Ren *et al.*, "Polar codes for arbitrary monotone chain rules on MACs," *IEEE Trans. Inf. Theory*, 2025.

[4] A. Hebbar *et al.*, "CRISP: Curriculum based sequential neural decoders for polar codes," in *Proc. IEEE ICC*, 2023.

[5] S. Cammerer *et al.*, "DeepPolar: Neural polar decoders," in *Proc. IEEE GLOBECOM*, 2024.